

1. Explicación de la funcionalidad

La funcionalidad desarrollada es la **Inteligencia Artificial de un enemigo (Agente Tanque)** dentro de un entorno multijugador online. Utilizando Unity ML-Agents, se ha entrenado a una red neuronal para que el tanque enemigo sea capaz de navegar por un mapa con obstáculos (muros), perseguir activamente al jugador y dispararle cuando tenga línea de visión, todo ello sincronizándose correctamente a través de un servidor Node.js sin causar conflictos de físicas entre el Host y el Cliente.

2. Proceso seguido con la IA y principales problemas encontrados

El desarrollo se realizó de forma iterativa, partiendo de una especificación inicial y usando la IA como asistente de código. Los principales problemas que encontramos y resolvimos fueron:

- **Problemas de Autoridad en Red:** Al principio, la IA se ejecutaba en los dos ordenadores a la vez (Host y Cliente), causando tirones. Se usó la IA para aislar las físicas y convertir al tanque del cliente en una marioneta.
- **El "Dilema del Muro" (Atascos):** La IA aprendió que alejarse del jugador restaba puntos. Por tanto, cuando había un muro en medio, prefería quedarse chocando contra él infinitamente antes que dar un rodeo.
- **Apariciones Bugeadas (Safe Spawn):** Al reiniciar el episodio de entrenamiento, la IA a veces aparecía dentro de las cajas de madera, volviéndose loca y arruinando el aprendizaje de la red neuronal.
- **Fuego Amigo en el Entrenamiento:** La IA aprendió a disparar muy rápido. Destruía al jugador en el segundo uno, haciendo desaparecer su objetivo y colapsando el script de ML-Agents por un error de variable nula.

3. Decisiones tomadas (Cambios en prompts y spec)

Para solucionar los problemas anteriores, no bastó con pedirle a la IA que "arreglara el código", sino que tuvimos que modificar activamente la especificación (Spec) y la estrategia:

1. **Cambio en el Spec de Recompensas:** Se modificó la especificación para que chocar contra un muro diera un castigo negativo ($-1f$), pero **sin finalizar el episodio** (`EndEpisode`). Esto obligó a la red neuronal a aprender a dar marcha atrás y maniobrar en lugar de rendirse.
2. **Nueva funcionalidad (Safe Spawn):** Se decidió implementar por prompt una función de "Radar" (`Physics2D.OverlapCircleAll`) para comprobar que las coordenadas de reaparición estuvieran vacías antes de colocar el tanque.
3. **Adición al Spec (Modo Entrenamiento):** Se añadió al `spec.md` la necesidad de un "Seguro de armas" (un booleano en el Inspector) para desactivar los disparos

durante la fase de entrenamiento, separando así el aprendizaje de movimiento del de combate.

4. Valoración crítica real

Respondiendo a las cuestiones clave sobre el uso de la IA en este SDD:

- **¿El agente ha seguido realmente la especificación?** Sí, a nivel de código estricto (sintaxis de C# y llamadas a la API de Unity), la IA fue extremadamente fiel a lo que se le pedía en el *spec*. Generó los sensores y las recompensas tal y como se definieron. Sin embargo, no fue capaz de prever las consecuencias lógicas de esas especificaciones en el comportamiento de la red neuronal.
- **¿Cuántas iteraciones han sido necesarias?** Han sido necesarias **4 iteraciones principales**. 1) Sincronización de red, 2) Corrección del castigo por distancia, 3) Implementación del Safe Spawn, 4) Bloqueo de armas para entrenamiento limpio. Además de pequeñas iteraciones de corrección de sintaxis (como el paso de *isKinematic* a *RigidbodyType2D* por el cambio a Unity 6).
- **¿Dónde falla más la IA (interpretación, ejecución, coherencia)?** La IA **falla estrepitosamente en la coherencia del ciclo de Machine Learning**. La IA es perfecta ejecutando código, pero carece de "sentido común". No fue capaz de deducir que si un tanque destruye a su objetivo, la variable *Transform objetivo* pasaría a ser nula y el código daría un error fatal. Ese contexto global (saber cómo funciona el juego completo y qué rompe un entrenamiento) tiene que ponerlo el humano.
- **¿Has tenido que modificar la especificación o solo los prompts?** Tuve que **modificar la especificación formal**. El plan inicial asimilaba que chocar contra un muro era un estado de fallo (fin del episodio), y eso impedía que la IA aprendiera a conducir. Modificar el *spec* para cambiar cómo se interpretaban las colisiones fue vital. Los prompts sirvieron para aplicar estos cambios al código, pero la dirección del proyecto la marcó el rediseño del *spec*.